

Experiment 6 | Lab Report

IMU Sensor Data Reading for Inertial Navigation: Acceleration and Angular Velocity

Reading a six-axis MPU6050 inertial measurement unit over I²C on a Raspberry Pi, and publishing it from an ESP32 as a live web dashboard

Submitted by	Chavan Atharva Sunil
Roll Number	240003021
Date of Experiment	10 September 2026
Boards Used	Raspberry Pi 4 (I ² C bus 1, GPIO2/GPIO3) and ESP32 DevKit (I ² C on GPIO21/GPIO22)

1 Aim

To read the three-axis **acceleration** and three-axis **angular velocity** of an **MPU6050** inertial measurement unit (IMU) over the I²C bus, and to relate these six quantities to what an inertial navigation system needs: the specific force from which velocity and position are integrated, and the turn rate from which orientation is tracked. The IMU is read first from a **Raspberry Pi**, printing the values to the terminal, and then from an **ESP32**, which serves them as a live web dashboard on its own Wi-Fi network while the sensor is moved by hand.

Objectives

- To connect a GY-521 (MPU6050) breakout to a Raspberry Pi and to an ESP32 over the two-wire I²C bus and identify it at its address 0x68.
- To read the accelerometer, gyroscope and temperature registers through a driver library on each board, in physical units (g, °/s, °C).
- To verify the readings against physics: at rest the acceleration vector has magnitude 1 g (gravity) and the gyroscope reads zero; when the sensor is tilted the 1 g redistributes between the axes, and when it is turned the gyroscope shows the rate of turn.
- To display the six values live in a browser served by the ESP32, and to discuss how they would be integrated into velocity, position and orientation — and why that integration drifts.

Brief Theory

An *inertial measurement unit* combines an accelerometer and a gyroscope, each with three orthogonal axes. The MPU6050 is a MEMS device: microscopic proof masses on silicon springs move under acceleration, and vibrating structures are deflected by the Coriolis force when the chip rotates; both displacements are read capacitively and digitised by 16-bit ADCs. The accelerometer full scale is selectable from ± 2 to ± 16 g and the gyroscope from ± 250 to ± 2000 °/s; the defaults used here, ± 2 g and ± 250 °/s, give resolutions of $61 \mu\text{g}$ and 0.0076 °/s per count. A temperature sensor is included for compensation.

What the two sensors measure

The accelerometer does *not* measure the acceleration of the sensor directly. It measures the *specific force* $\mathbf{f} = \mathbf{a} - \mathbf{g}$: the sum of the sensor's own acceleration and the reaction to gravity. A sensor lying still on a table therefore reads +1 g on the axis pointing up and zero on the other two, and the magnitude

$$|\mathbf{f}| = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

is 1 g in *any* orientation as long as the sensor is not accelerating. Tilting the sensor merely redistributes that 1 g between the axes, which is what makes the accelerometer a tilt sensor:

$$\text{pitch} = \arctan \frac{a_x}{\sqrt{a_y^2 + a_z^2}}, \quad \text{roll} = \arctan \frac{a_y}{\sqrt{a_x^2 + a_z^2}}.$$

The gyroscope measures angular velocity ω about each axis in $^\circ/\text{s}$: zero when the sensor is still, whatever its orientation, and non-zero only while it is turning.

From IMU readings to navigation

Inertial navigation is the process of integrating these six quantities:

$$\theta(t) = \int \omega dt, \quad \mathbf{v}(t) = \int (R(\theta) \mathbf{f} + \mathbf{g}) dt, \quad \mathbf{p}(t) = \int \mathbf{v} dt.$$

The gyroscope is integrated once to track the orientation θ ; that orientation is used to rotate the specific force into the world frame and remove gravity; the result is integrated once for velocity and again for position. Every integration also integrates the sensor's errors — a constant gyro bias becomes a steadily growing angle error, and an accelerometer bias of only 1 mg becomes a position error growing as $\frac{1}{2}bt^2$, about 18 m after one minute — so an IMU alone is only useful for short intervals and is always combined with an absolute reference (GPS, a barometer, a magnetometer, star trackers on a spacecraft) that resets the drift.

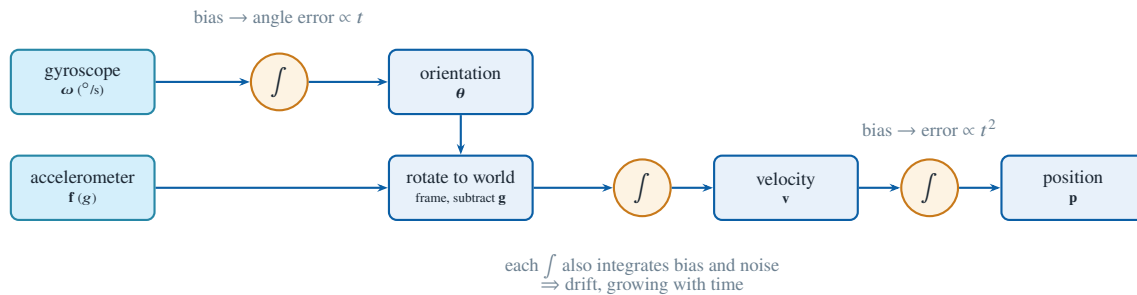


Figure 1: The inertial-navigation chain. This experiment produces the two inputs on the left; the discussion returns to why the integrations on the right drift.

The module. The sensor came as a **GY-521** breakout: the MPU6050 with its own 3.3 V regulator (so VCC accepts 3.3 or 5 V), pull-up resistors on SDA and SCL, and the AD0 address pin pulled LOW, which sets the I²C address to 0x68. Only four of its eight pins are needed: VCC, GND, SDA and SCL. The bus pins are the same ones as in Experiment 5 — GPIO2/GPIO3 on the Raspberry Pi (I²C must be enabled) and GPIO21/GPIO22 on the ESP32.

2 Materials Required

#	Component	Qty.	Purpose in the setup
1	Raspberry Pi 4 (in case, with SD card, OS installed)	1	Part A host; I ² C master, prints readings to the terminal (also powers the ESP32 over USB in Part B)
2	ESP32 DevKit (30-pin)	1	Part B host; I ² C master, Wi-Fi access point and web server
3	GY-521 breakout with MPU6050 (6-axis IMU)	1	Measures acceleration and angular velocity; shared between the two parts
4	Female–female jumper wires	4	VCC, GND, SDA and SCL between header and module
5	Micro-USB cable, laptop with Arduino IDE, and a tablet or phone with Wi-Fi	1	Programming the ESP32 and viewing the dashboard
6	USB-C power supply, monitor, keyboard and mouse	1	Running and observing the Raspberry Pi
7	Software: adafruit-circuitpython-mpu6050 (Pi); Adafruit MPU6050 library and ESP32 core (Arduino IDE)	—	Sensor drivers on the two boards

3 Methodology

The same GY-521 module was used on both boards, wired the same way each time: four jumper wires for VCC, GND, SDA and SCL. The module's other pins (XDA, XCL, AD0, INT) were left unconnected, which leaves it in plain I²C mode at 0x68. No breadboard or extra components were needed because the pull-up resistors and the regulator are on the module.

3.1 Connections

Module pin	Raspberry Pi 4 (Part A)	ESP32 (Part B)	Function
VCC	Pin 1 — 3.3 V	3V3 (red)	Supply, through the module's regulator
GND	Pin 6 — GND	GND (black)	Ground
SDA	Pin 3 — GPIO2 (board.SDA)	GPIO21 (green)	Serial data, bidirectional
SCL	Pin 5 — GPIO3 (board.SCL)	GPIO22 (yellow)	Serial clock from the master
XDA, XCL, AD0, INT	not connected	not connected	Auxiliary bus, address select (LOW ⇒ 0x68), interrupt

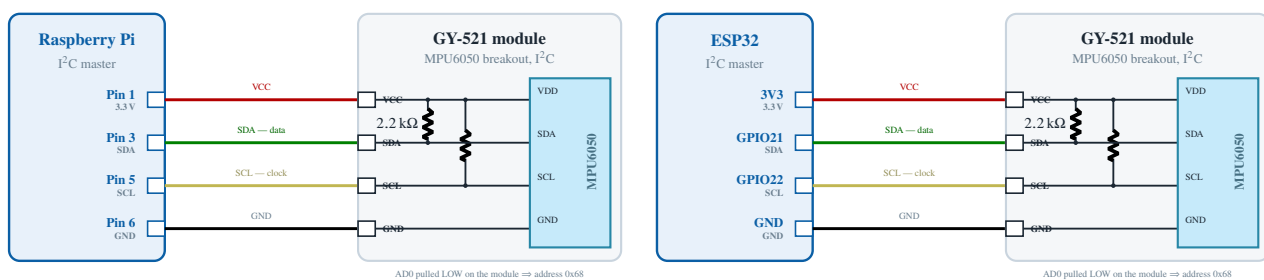


Figure 2: Circuit schematics for the two parts. Four wires each; the pull-ups and the address-setting AD0 resistor are on the GY-521 module. Wire colours as used in Part B.

3.2 Part A — Raspberry Pi, readings on the terminal

With I²C enabled, the sensor was confirmed on the bus with `i2cdetect -y 1` and the Adafruit CircuitPython driver installed:

Listing 1: Preparing the Raspberry Pi.

```

1 sudo raspi-config      # Interface Options -> I2C -> Enable
2 sudo apt install i2c-tools
3 i2cdetect -y 1         # the MPU6050 shows up at 0x68
4 pip3 install adafruit-circuitpython-mpu6050

```

The program (Listing 2) opens the bus, creates the sensor object, and loops twice a second reading the acceleration and gyro tuples (in m/s^2 and rad/s , which it converts to g and $^\circ/\text{s}$ to match the ESP32 dashboard), plus the temperature, and computes the magnitude $|\mathbf{f}|$ and the tilt angles so that the rest test of the theory section can be checked at a glance.

Listing 2: Part A — imu.py on the Raspberry Pi: acceleration, angular velocity, tilt and temperature twice a second.

```

1 import time
2 import math
3 import board
4 import busio
5 import adafruit_mpu6050
6
7 G = 9.80665          # m/s^2 per g
8
9 # Initialize I2C and the sensor (GY-521, AD0 low -> 0x68)
10 i2c = busio.I2C(board.SCL, board.SDA)
11 mpu = adafruit_mpu6050.MPU6050(i2c, address=0x68)
12 mpu.accelerometer_range = adafruit_mpu6050.Range.RANGE_2_G
13 mpu.gyro_range = adafruit_mpu6050.GyroRange.RANGE_250_DPS
14
15 while True:
16     ax, ay, az = (v / G for v in mpu.acceleration)    # g
17     gx, gy, gz = (math.degrees(v) for v in mpu.gyro)   # deg/s
18     mag = math.sqrt(ax*ax + ay*ay + az*az)             # should be ~1 g at rest
19     pitch = math.degrees(math.atan2(ax, math.sqrt(ay*ay + az*az)))
20     roll = math.degrees(math.atan2(ay, math.sqrt(ax*ax + az*az)))
21
22     print("-----")
23     print(f"Accel (g) : X {ax:6.2f} Y {ay:6.2f} Z {az:6.2f} |a| {mag:4.2f}")
24     print(f"Gyro (deg/s): X {gx:7.2f} Y {gy:7.2f} Z {gz:7.2f}")
25     print(f"Tilt (deg) : pitch {pitch:6.1f} roll {roll:6.1f}")
26     print(f"Temperature : {mpu.temperature:.2f} C")
27
28     time.sleep(0.5)

```

3.3 Part B — ESP32, readings on a web dashboard

On the ESP32 the sensor is read with the Adafruit MPU6050 Arduino library. As in Experiment 5, the sketch (Listing 3) starts the ESP32 as a Wi-Fi **access point** and runs a web server: `/` returns the dashboard page, and `/data` returns the seven current readings as JSON. The page — titled “ESP32 IMU Dashboard” — has three groups of cards, *Accelerometer* (X, Y, Z in g), *Gyroscope* (X, Y, Z in °/s) and *Temperature*, and a few lines of JavaScript refresh them from `/data` every 200 ms, fast enough to follow the sensor while it is turned by hand. A tablet joined the ESP32’s network and opened `http://192.168.4.1`.

Listing 3: Part B — ESP32 sketch: MPU6050 over I²C, served as a live IMU dashboard from the board’s own access point.

```

1 #include <WiFi.h>
2 #include <WebServer.h>
3 #include <Wire.h>
4 #include <Adafruit_MPU6050.h>
5 #include <Adafruit_Sensor.h>
6
7 const char *AP_SSID = "ESP32-IMU";    // the ESP32 creates this Wi-Fi network
8 const char *AP_PASS = "12345678";
9 const float G = 9.80665;              // m/s^2 per g
10
11 Adafruit_MPU6050 mpu;                 // I2C, default pins SDA = 21, SCL = 22
12 WebServer server(80);
13
14 // The dashboard page. Its script asks the ESP32 for fresh values 5 times a second.
15 const char PAGE[] PROGMEM = R"html(
16 <!DOCTYPE html><html><head><meta charset="utf-8">
17 <meta name="viewport" content="width=device-width,initial-scale=1">
18 <title>ESP32 IMU Dashboard</title>
19 <style>
20 body{font-family:sans-serif;background:#1c2738;color:#ddd;margin:16px}
21 h2{font-size:15px;margin:14px 0 6px} .row{display:flex;gap:10px}
22 .card{flex:1;background:#2b3a50;border-radius:10px;padding:8px;text-align:center}
23 .l{font-size:12px;color:#aaa} .v{font-size:26px;font-weight:bold}
24 .a.v{color:#4fc3f7} .g.v{color:#b39ddb} .t.v{color:#ff9800}
25 </style></head><body>
26 <h2>Accelerometer</h2><div class="row a">
27 <div class="card"><div class="l">X Axis</div><div class="v" id="ax">--</div><div class="l">g</div></div>
28 <div class="card"><div class="l">Y Axis</div><div class="v" id="ay">--</div><div class="l">g</div></div>
29 <div class="card"><div class="l">Z Axis</div><div class="v" id="az">--</div><div class="l">g</div></div></div>
30 <h2>Gyroscope</h2><div class="row g">
31 <div class="card"><div class="l">X Axis</div><div class="v" id="gx">--</div><div class="l">&deg;/s</div></div>
32 <div class="card"><div class="l">Y Axis</div><div class="v" id="gy">--</div><div class="l">&deg;/s</div></div>
33 <div class="card"><div class="l">Z Axis</div><div class="v" id="gz">--</div><div class="l">&deg;/s</div></div></div>
34 <h2>Temperature</h2><div class="row t">
35 <div class="card"><div class="l">MPU6050 Temperature</div><div class="v" id="t">--</div><div class="l">&deg;C</div>
36 </div></div>
37 <script>
38 setInterval(async () => {
39     const d = await (await fetch('/data')).json();
40     for (const k of ['ax','ay','az','gx','gy','gz','t'])
41         document.getElementById(k).textContent = d[k].toFixed(2);

```

```

41 }, 200);
42 </script></body></html>)html";
43
44 void handleRoot() { server.send_P(200, "text/html", PAGE); }
45
46 void handleData() { // the seven readings as one JSON object
47     sensors_event_t a, g, t;
48     mpu.getEvent(&a, &g, &t); // m/s^2, rad/s, degC
49     String json = "{";
50     json += "\"ax\":" + String(a.acceleration.x / G, 2);
51     json += "\"ay\":" + String(a.acceleration.y / G, 2);
52     json += "\"az\":" + String(a.acceleration.z / G, 2);
53     json += "\"gx\":" + String(g.gyro.x * 180.0 / PI, 2);
54     json += "\"gy\":" + String(g.gyro.y * 180.0 / PI, 2);
55     json += "\"gz\":" + String(g.gyro.z * 180.0 / PI, 2);
56     json += "\"t\":" + String(t.temperature, 2) + "}";
57     server.send(200, "application/json", json);
58 }
59
60 void setup() {
61     Serial.begin(115200);
62     Wire.begin(21, 22); // SDA, SCL
63     if (!mpu.begin(0x68)) {
64         Serial.println("MPU6050 not found - check the wiring and the address");
65         while (true) delay(10);
66     }
67     mpu.setAccelerometerRange(MPU6050_RANGE_2_G);
68     mpu.setGyroRange(MPU6050_RANGE_250_DEG);
69     mpu.setFilterBandwidth(MPU6050_BAND_21_HZ); // on-chip low-pass filter
70     WiFi.softAP(AP_SSID, AP_PASS); // start the access point
71     Serial.print("Dashboard at http://");
72     Serial.println(WiFi.softAPIP()); // 192.168.4.1
73     server.on("/", handleRoot);
74     server.on("/data", handleData);
75     server.begin();
76 }
77
78 void loop() { server.handleClient(); }

```

3.4 Procedure

- Part A.** With the Pi off, the module's VCC, GND, SDA and SCL were wired to header pins 1, 6, 3 and 5. I²C was enabled in raspi-config, and i2cdetect confirmed a device at 0x68.
- The driver was installed and Listing 2 was saved as imu.py and run with python3 imu.py. The terminal printed the six IMU values, the magnitude, the tilt angles and the temperature twice a second, first with the module flat on the desk and then while it was tilted and turned.
- Part B.** The same module was moved to the ESP32: VCC to 3V3, GND to GND, SDA to GPIO21, SCL to GPIO22. The ESP32 was powered from a USB port of the Raspberry Pi. In the Arduino IDE the ESP32 board package and the Adafruit MPU6050 library (with its Unified Sensor and BusIO dependencies) were installed and Listing 3 was uploaded.
- A tablet joined the ESP32's network and opened http://192.168.4.1. The dashboard appeared with the seven values updating live. The module was held in the hand and tilted and rotated in front of the tablet while the screen was recorded, then set down at rest.
- Readings were transcribed from the recordings, and the acceleration magnitude and gyroscope readings were checked against the rest and motion expectations of the theory section.

4 Result and Output

Both boards read the MPU6050 correctly over I²C. On the **ESP32** the dashboard at 192.168.4.1 followed the sensor in real time: **at rest** the acceleration components summed to a magnitude of 1.03–1.05 g and the gyroscope read within $\pm 5^\circ/\text{s}$ of zero, while **during hand movement** the magnitude departed from 1 g (as low as 0.83 and as high as 1.65 g) and the gyroscope showed turn rates of up to $\pm 180^\circ/\text{s}$. The chip temperature held at 24.9–25.1 °C. The **Raspberry Pi** printed the same quantities to its terminal, with the tilt angles derived from the accelerometer.

Observed outputs — Part B, ESP32 dashboard

Clip time	Accelerometer (g)			Gyroscope ($^\circ/\text{s}$)			f (g)	State
	X	Y	Z	X	Y	Z		
0 s	-0.05	0.03	-1.03	-193.7	-39.4	-29.9	1.03	flat on the desk, being picked up
2 s	0.15	0.25	-0.80	-13.2	28.6	-44.3	0.85	in the hand, moving
4 s	0.00	0.70	-0.45	141.2	58.9	-101.7	0.83	being tilted
5 s	-0.63	0.95	0.94	79.5	103.2	23.8	1.48	turning quickly
7 s	0.92	0.85	-0.35	-36.1	-107.4	92.7	1.30	turning
9 s	-0.77	1.38	0.48	-153.2	-177.0	-34.6	1.65	fastest movement
11 s	-0.10	0.83	-0.69	73.9	-29.9	5.5	1.08	slowing
12 s	-0.06	0.55	-0.70	101.0	-68.5	-13.7	0.89	slowing
14.8 s	-0.29	0.47	-0.89	-4.8	2.7	2.6	1.05	at rest on the desk, tilted

Values read from the dashboard in the recording; |f| is computed from the three accelerometer values. The sign of Z is negative when the module lies component-side down, as it did here. The dashboard's temperature card read between 24.91 and 25.05 °C throughout.

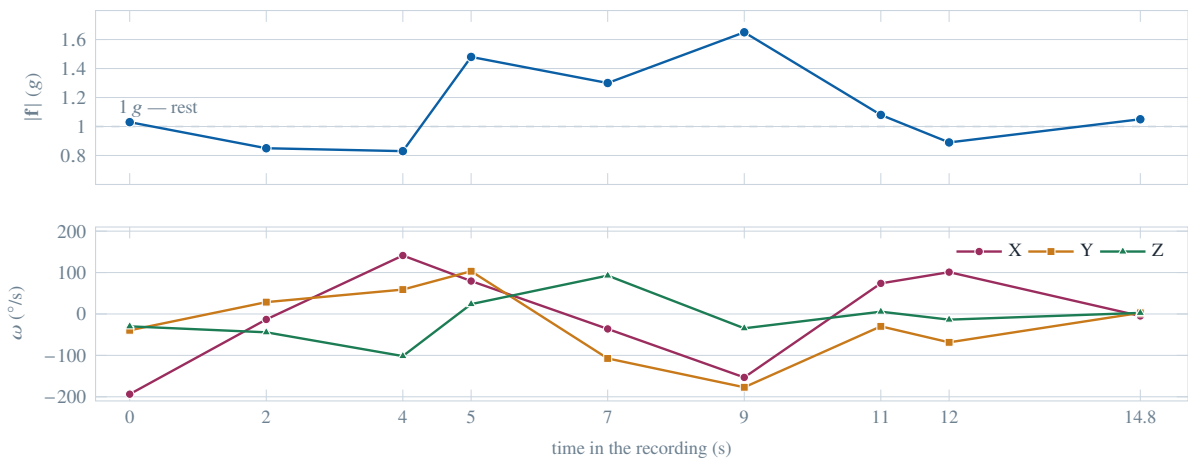


Figure 3: The Part B readings over the recording. *Top:* the magnitude of the accelerometer vector sits at 1 g at the start and end, when the sensor is still, and swings away from it while the sensor is being moved — the difference is the sensor's own acceleration. *Bottom:* the three gyroscope rates, large and changing sign while the sensor is turned by hand, all within $\pm 5^\circ/\text{s}$ once it is set down.

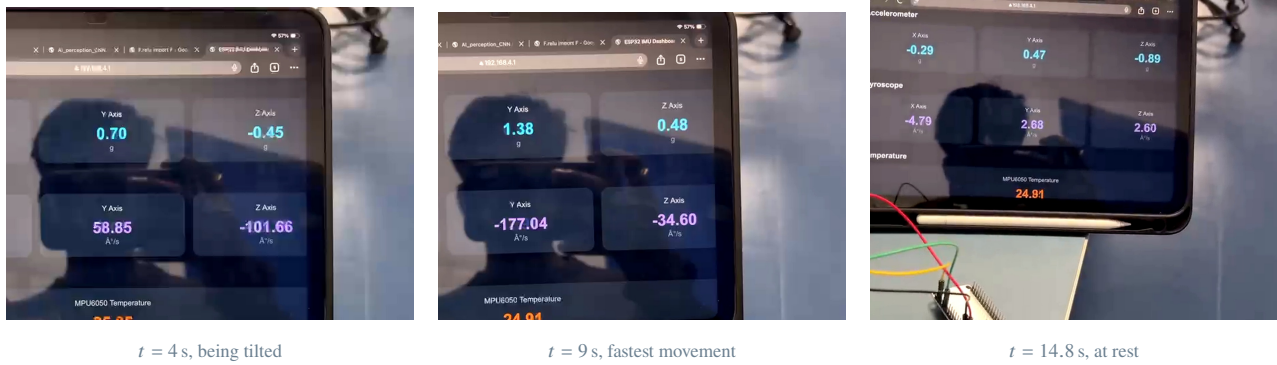


Figure 4: Part B. The “ESP32 IMU Dashboard” served at 192.168.4.1, viewed on a tablet: three accelerometer cards in g , three gyroscope cards in $^{\circ}/s$ and the chip temperature. In the last frame the sensor has been set down: the gyroscope is near zero and the accelerometer shows the 1 g of gravity split over the axes by the tilt of the desk-top module. (The “Å” before the units is the same encoding slip as in Experiment 5, fixed in Listing 3 by declaring the charset.)

Checking the rest reading against the theory

For the final frame, $(a_x, a_y, a_z) = (-0.29, 0.47, -0.89) \text{ g}$:

$$|\mathbf{f}| = \sqrt{0.29^2 + 0.47^2 + 0.89^2} = 1.05 \text{ g}, \quad \text{pitch} = \arctan \frac{-0.29}{\sqrt{0.47^2 + 0.89^2}} = -16^{\circ}, \quad \text{roll} = \arctan \frac{0.47}{\sqrt{0.29^2 + 0.89^2}} = 27^{\circ}.$$

The magnitude is 1 g to within the $\pm 5\%$ that the uncalibrated $\pm 2 \text{ g}$ range of the MPU6050 typically gives, so the sensor was indeed at rest, resting at a tilt of about 16° and 27° about its two horizontal axes — it was lying on the bundle of jumper wires, not flat. The gyroscope’s $-4.8, 2.7$ and $2.6^{\circ}/s$ are its zero-rate offsets, which a real application measures at start-up and subtracts.

Observed outputs — Part A, Raspberry Pi terminal

[To be completed.] The Raspberry Pi run is to be recorded and its terminal readings — module flat on the desk, then tilted, then turned — transcribed here in the same form as the ESP32 table, together with a terminal still. The program and the wiring of Part A are complete above.

The setup

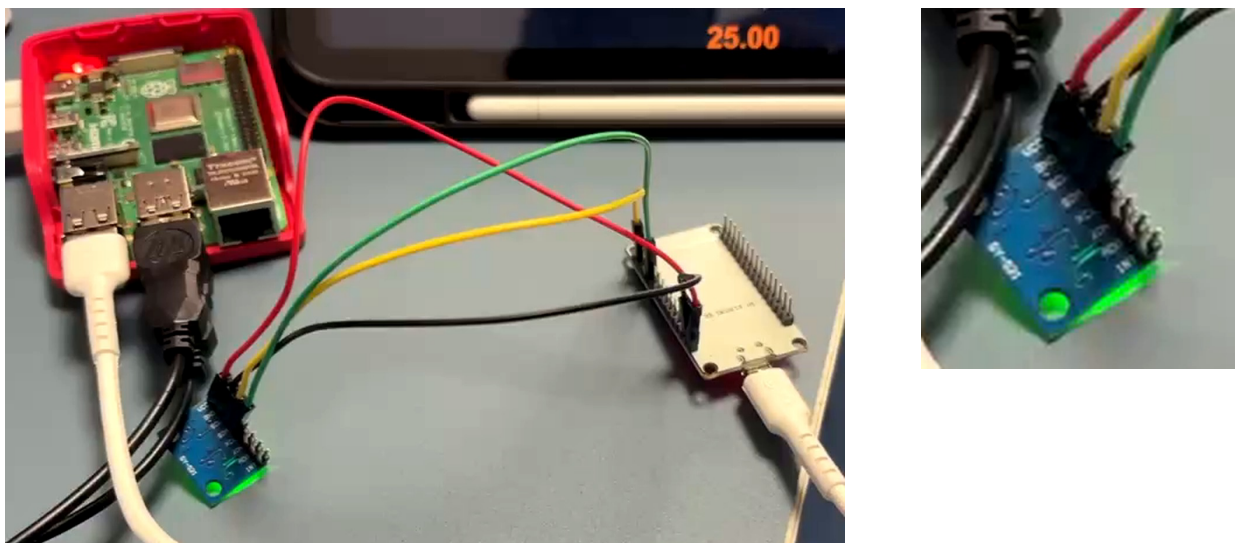


Figure 5: Left: Part B — the ESP32 DevKit (right, powered from a USB port of the Raspberry Pi) with the GY-521 on four jumper wires — red VCC, black GND, green SDA, yellow SCL — in front of the tablet showing the dashboard. Right: the GY-521 module with its power LED lit; the MPU6050 is the small square package in the centre.

Video demonstration and code

Google Drive link

The demonstration videos, this report and the complete source code are available in the following shared folder:

<https://drive.google.com/drive/folders/1cdR2mrryf5j0LEnQgIgYEAs-zB0Qj0jO?usp=sharing>

Access has been set to *anyone with the link can view*.

Discussion

- **The accelerometer measures gravity too.** The clearest result of the experiment is that a motionless sensor reads 1 g , not zero. Only the *deviation* of $|\mathbf{f}|$ from 1 g in Figure 3 — down to 0.83 g and up to 1.65 g while the module was swung — is the sensor's own acceleration, and separating the two properly needs the orientation from the gyroscope, exactly as the chain in Figure 1 shows.
- **Tilt from the accelerometer, turn rate from the gyroscope.** The rest reading gave the tilt of the module to within a degree or two without any integration at all, because gravity is a fixed reference; the gyroscope, by contrast, only ever gives a *rate*, and its reading returns to (nearly) zero the moment the motion stops. The two sensors are complementary: the accelerometer is drift-free but useless while the sensor is accelerating, the gyroscope is fast and unaffected by acceleration but drifts. Practical orientation filters (complementary or Kalman filters) blend them.
- **Why an IMU alone cannot navigate for long.** The residual gyroscope offsets of $2.6\text{--}4.8^\circ/\text{s}$ at rest, if integrated, would rotate the estimated orientation by a full turn in under two minutes; the accelerometer's 5% scale error, integrated twice, would place the sensor tens of metres away within a minute. Calibration at start-up (subtracting the zero-rate offsets, scaling $|\mathbf{f}|$ to exactly 1 g) reduces this, but not to zero, so inertial navigation always pairs the IMU with an external fix — GPS or a barometer on the ground, star trackers and ground tracking on a spacecraft.
- **Sampling and filtering.** The dashboard refreshes five times a second, which is enough to watch but far too slow for navigation; the MPU6050 itself can deliver up to 1 kHz and has an on-chip low-pass filter (`setFilterBandwidth`) that trades noise for response time. A navigation loop would read it from a timer interrupt at a fixed rate, since the integrals depend on knowing dt exactly.
- **Two ways to deliver the data.** As in Experiment 5, the Pi prints to a terminal while the ESP32 serves a web page from its own access point. The IMU shows the advantage of the second approach: with the sensor in the hand and the tablet on the desk, the live values could be watched while the module was moved, which a terminal on a fixed monitor made awkward.

5 Conclusion

The MPU6050 IMU was read over I²C from both a Raspberry Pi and an ESP32, giving three-axis acceleration and three-axis angular velocity in physical units. The ESP32 served the six values as a live dashboard on its own Wi-Fi network, and the recording showed the behaviour the theory predicts: a resting sensor reads a 1 g acceleration vector and a zero turn rate, tilting redistributes the 1 g between the axes (from which a tilt of 16° and 27° was recovered), and moving the sensor by hand produced accelerations of up to 1.65 g and turn rates of up to $180^\circ/\text{s}$.

These six quantities are the raw inputs of inertial navigation. The experiment also showed its central difficulty: velocity, position and orientation are obtained by integration, and the small biases seen at rest — a few $^\circ/\text{s}$ of gyroscope offset, a few per cent of accelerometer scale — grow without bound once integrated, which is why an IMU is always used together with an absolute reference.